Key Design Decisions

A. How consumable items trigger their effects

Design 1: Create a separate Action class for each consumable item (e.g., ConsumeAppleAction, ConsumeCookiesAction, ConsumeFlaskAction). Each action knows the specific item type and directly manipulates the actor's state.

Pros	Cons
Each action is self-contained and easy to understand in isolation.	Leads to class proliferation; every new consumable requires a new Action class.
No abstraction overhead; no interface to define.	Violates the Open-Closed Principle — adding a new consumable requires creating a new concrete Action class and wiring it in.
	Duplicates structural logic (removing from inventory, printing feedback) across multiple action classes, violating DRY.

Design 2: Define a Consumable interface with consumedBy(Actor, GameMap) and getConsumeMenuDescription(Actor). A single ConsumeAction accepts any Consumable and delegates it to consumedBy() in its execute() method.

Pros	Cons
Adding a new consumable only requires implementing the Consumable interface — no new action class needed (Open-Closed Principle).	Requires defining an additional interface, introducing a small amount of structural overhead.
ConsumeAction depends on the Consumable abstraction rather than on concrete item classes (Dependency Inversion Principle).	Each consumable must implement its own effect logic, distributing behaviour across classes rather than centralising it.
Eliminates duplicated action logic; structural concerns (menu description, delegation) exist in only one place (DRY).	
Each item encapsulates its own consumption effect, adhering to the Single Responsibility Principle.	

B. How sterilisation affects consumable behaviour

Design 1: In each consumable's consumedBy() method, use instanceof SterilisationBox to check if the actor's inventory contains a sterilisation box.

Pros	Cons
Directly checks for the concrete item, which is easy to follow.	Creates tight coupling between every consumable and the SterilisationBox class.
	If a second item provides sterilisation (e.g., a UV light), every consumable must be updated to check for it, violating Open-Closed Principle.
	Requires iterating through the inventory manually, duplicating lookup logic.

Design 2: SterilisationBox enables Ability.HAS_STERILISATION on itself. Consumables check actor.hasAbility(Ability.HAS_STERILISATION) to decide their effect. The engine's Actor.hasAbility() automatically scans inventory items.

Pros	Cons
Consumables are completely decoupled from SterilisationBox; both depend on the abstract Ability enum (Dependency Inversion Principle).	The indirection through abilities may be less obvious to someone unfamiliar with the engine's capability scanning mechanism.
Any future item that grants HAS_STERILISATION will work with all consumables automatically (Open-Closed Principle).	
No manual inventory iteration needed; the engine handles scanning.	

C. How status effects (poison and burn) are implemented

Design 1: Track damage-over-time using ad-hoc counters inside the affected item or actor class. For example, Apple could set a field on ContractedWorker like poisonTurnsRemaining = 5.

Pros	Cons
No additional classes required; just integer fields on existing classes.	Mixes status-tracking responsibility into classes that should not own it (violates Single Responsibility Principle).
	Cannot support multiple simultaneous effects or stacking without increasingly complex logic inside the actor.
	Requires the actor to know about all possible status effects, violating Open-Closed Principle.

Design 2: Implement PoisonStatus and BurnStatus as classes implementing the engine's Status interface. Each status tracks its own damage and duration via tickStatus(GameEntity, Location), and is ticked automatically by the engine's GameMap.tick().

Pros	Cons
Each status effect is a self-contained class with a single responsibility — tracking and applying its own damage per turn.	Creates additional classes (one per status type).
Multiple status effects can coexist independently on the same actor, and multiple instances of the same type can stack naturally.	Requires understanding the engine's automatic status-ticking mechanism.
New status types can be added without modifying any existing class (Open-Closed Principle).	
Leverages the engine's existing Status lifecycle (isStatusActive(), tickStatus()), avoiding reinventing timer logic (DRY).	

D. How the Lantern's fire-spawning mechanism works

Design 1: When the Lantern leaks, directly damage the actor and print a message. No ground modification.

Pros	Cons
Minimal implementation; no new ground class needed.	Does not satisfy the specification requirement that fire spawns on the ground tile.
	Other actors walking over the same tile would not be affected, missing the environmental hazard aspect.

Design 2: The Lantern's tick(Location, Actor) spawns a Fire ground tile, which stores the original ground and reverts to it after 5 turns. Fire.tick(Location) applies BurnStatus to any actor standing on it each turn.

Pros	Cons
Fire is a self-contained ground type that manages its own lifetime and effect application (Single Responsibility Principle).	Requires storing a reference to the original ground for reversion, adding a composition relationship.
The fire affects any actor on the tile, not just the lantern carrier — correctly modelling an environmental hazard.	
The Lantern only creates the fire; it does not manage the fire's lifecycle. Responsibilities are cleanly separated between Lantern (probability, oil management) and Fire (duration, damage, reversion).	
Using composition (Fire stores originalGround) avoids inheritance complexities for temporary ground state.	

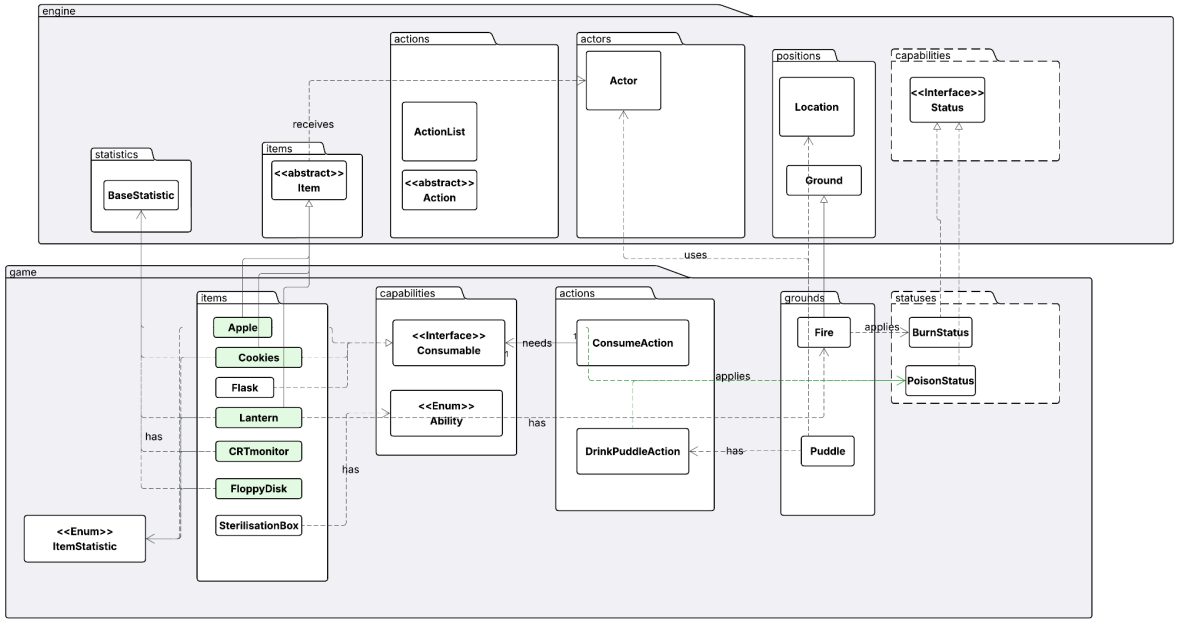
E. How Puddle interaction is offered to the player

Design 1: Check in ContractedWorker.playTurn() whether the actor is standing on a Puddle (via display character or instanceof), and manually add a drink action.

Pros	Cons
Explicit control over when the drink option appears.	Couples ContractedWorker to the Puddle class, violating Dependency Inversion.
	Every new ground interaction would require modifying ContractedWorker, violating Open-Closed Principle.
	Duplicates the engine's existing ground action-collection mechanism.

Design 2: Puddle overrides Ground.allowableActions(Actor, Location, String) and returns a DrinkPuddleAction when the actor is standing on it (direction is empty string). The engine's World.prepareAllowableActions() automatically collects this.

Pros	Cons
Follows the same pattern as all other ground interactions in the engine, maintaining consistency.	DrinkPuddleAction is specific to Puddle and cannot be reused for other liquid sources without modification.
ContractedWorker remains unaware of Puddle, maintaining low coupling.	
New ground interactions can be added by simply overriding allowableActions() in new ground types (Open-Closed Principle).	



Summary

The final design of requirement 2 utilises Design 2 of parts A, B, C, D, and E stated above to implement the consumable items, sterilisation mechanic, status effects, fire hazard, and puddle drinking interaction.

The Consumable interface abstracts the consumption contract into consumedBy(Actor, GameMap) and getConsumeMenuDescription(Actor). Apple, Cookies, and Flask all implement this interface, encapsulating their individual effects while sharing a single ConsumeAction that depends only on the abstraction (Dependency Inversion Principle). This eliminates the need for per-item action classes and ensures that new consumables can be added by simply implementing the interface (Open-Closed Principle).

Sterilisation is mediated through the Ability.HAS_STERILISATION enum value, decoupling SterilisationBox from all consumables. The engine's Actor.hasAbility() scans inventory items automatically, so consumables need only check for the ability rather than searching for a specific item type. This enables future sterilisation sources without modifying any consumable class.

PoisonStatus and BurnStatus implement the engine's Status interface, each managing their own damage-per-turn counters and duration. The engine ticks all statuses automatically via GameMap.tick(), allowing multiple effects to coexist and stack independently on the same actor without any centralised management.

The Lantern spawns a Fire ground tile via its tick(Location, Actor) method, cleanly separating the probability/oil logic (Lantern) from the duration/damage/reversion logic (Fire). Fire uses composition to store the originalGround, reverting the tile when its duration expires. This design models fire as a true environmental hazard that persists independently of the item that created it.

Puddle overrides Ground.allowableActions() to offer DrinkPuddleAction, leveraging the engine's existing action-collection infrastructure rather than hardcoding interactions in the

actor. DrinkPuddleAction checks Ability.HAS_STERILISATION to determine whether the water heals or poisons, maintaining consistency with the sterilisation pattern used by item consumables.